

CAT 1

Hello World! (in Vue.js)

Alejandro Pérez Bueno

Mar 18, 2026

Table of Contents

Exercise 1	2
Exercise 2	2
Exercise 3	3
Exercise 4	3
Exercise 5	4
Exercise 6	4
Exercise 7	5
Exercise 8	5
Exercise 9	6
Exercise 10	7
Exercise 11	7
Exercise 12	8
Exercise 13	8
Exercise 14	8
Exercise 15	9
Exercise 16	9
Exercise 17	9
Exercise 18	10
Exercise 19	10
References	11

Exercise 1

JavaScript has an interesting history. It originally started as a simple scripting language meant just for browsers, but today it holds a monopoly in the space. This change took place in the early 2000s with the introduction of Ajax (Asynchronous JavaScript and XML) thanks to its asynchronous server communication. Developers could now update their sites without needing a full-page reload ([Brown 2016, chap. 18](#)). This change alone is probably responsible for JavaScript being at the core of modern web apps.

As web development got more complex, programmers needed better tools to manage their code. Originally jQuery was quite popular because it helped developers edit the DOM and handle browser differences. However, jQuery is considered outdated for building modern web applications today ([Brown 2016, chap. 18](#)).

Instead, modern solutions like React, Angular, and Vue took over. These newer frameworks come with features that help with packaging code. They also run code through transcompilers to ensure modern syntax works smoothly on older browsers ([Brown 2016, chap. 2](#)). Also, these tools have changed how we organize code by splitting the user interface into reusable components.

These frameworks are the right foundation that is required for building Single Page Applications (SPAs). An SPA is a web app that loads a single HTML document and updates the content dynamically using JavaScript as people interact with it ([MDN Contributors 2025](#)). Before SPAs became the norm, older websites forced the server to load a brand new page for every single click. That caused annoying screen flickering and slower load times ([OutSystems 2025](#)). SPAs fixed this by moving the rendering work to the browser and grabbing only the data they need through APIs. This shift radically changed web development because it split frontend and backend teams, cut down on server stress and made web apps feel as fast and smooth as native phone apps ([Adobe 2023](#)).

Exercise 2

The virtual DOM (VDOM) is a programming concept where a lightweight representation of the actual UI is maintained in a JavaScript object. Manipulating the real DOM is often slow and resource heavy. By using a virtual DOM, frameworks can optimize performance by avoiding full DOM repaints every time the state of an application changes ([GeeksforGeeks 2026](#)).

Both React and Vue rely on the virtual DOM to keep applications fast, but they handle the underlying mechanics through slightly different processes.

React represents the UI as a tree of js objects. When the application state updates, React builds a new VDOM tree, which it then compares to the previous version using a “diffing” algorithm. Then React uses these differences between the two trees to apply only the necessary changes to the real DOM, minimizing manipulation costs ([GeeksforGeeks 2026](#)).

Vue also uses the VDOM but uses a hybrid approach: “Compiler-Informed Virtual DOM” ([Omoyeni 2024; Vue.js 2025](#)). During the build step Vue compiles its templates into optimized render functions that output virtual DOM trees. When data changes, Vue creates a new tree and patches the differences into the actual

DOM ([Vue.js 2025](#)).

Even with the Vue optimizations, the VDOM still requires memory to store the virtual nodes and processing power to traverse the tree and perform reconciliation ([Omoyeni 2024](#)). In order to solve this, Vue 3 introduces Vapor Mode, which completely eliminates the VDOM abstraction ([Batsou 2025](#)). Inspired by frameworks like Solid.js, it relies entirely on compile time analysis to generate direct DOM manipulation code for reactivity updates ([Omoyeni 2024](#); [Rüsche 2025](#)). Vapor Mode compiled output generates code that manipulates the specific DOM elements that need updating directly, rather than creating a new tree and comparing it with an old one like VDOM does. Vapor mode is also lighter and consumes less memory, due to the fact that there is no VDOM to maintain ([Batsou 2025](#)).

Exercise 3

In SSR the server processes the request, fetches data and compiles the templates to send an HTML page to the client ([SolutionsHub 2024](#)). This traditional method gives better SEO performance because search engine crawlers receive full content immediately, and also ensures faster initial page loads, improving the user experience. However, SSR places a heavy computational load on the server and users may experience a brief confusing delay in responsiveness until the JavaScript executes on the client. SSR is best suited for websites that focus on content, such as blogs or e-commerce platforms.

CSR on the other hand shifts the rendering workload to the client devices. The browser downloads a minimal HTML file and the JavaScript required to construct the page dynamically ([Next.js 2026](#)). After this future interactions are much faster than SSR. This approach significantly reduces server load. Contrary to SSR, since search engines initially see a largely empty HTML page, SSR is worse. Users may also experience slow initial load times while the browser downloads and parses the necessary large JavaScript files before content appears ([Pluralsight 2019](#)). One should use CSR for highly interactive SPAs.

Modern development is increasingly moving toward hybrid approaches where SSR handles the crucial initial load for performance and SEO, and CSR takes over for subsequent interactive user sessions ([Shopify 2025](#)).

Exercise 4

Semantic HTML is writing HTML using tags that describe the meaning of the content inside them so that humans and machines can read it ([freeCodeCamp 2019](#)). In the past, people used `<div>` and `<span>` tags all the time to build layouts. Those are non-semantic because they do not say much about what the content actually does or means. Semantic HTML uses specific tags that clearly define the purpose of a section ([Pavlik 2025](#)).

One clear benefit of using semantic HTML is accessibility. Screen readers for instance rely on semantic layouts to figure out page structure, which helps visually impaired users navigate websites easily and jump directly to important places in the app ([Pavlik 2025](#); [MDN Web Docs 2025a](#)). It also helps with SEO ([Montti 2024](#)), and makes the code easier for developers to read and maintain ([freeCodeCamp 2019](#)).

Here some common semantic elements and what they do:

- **<header>**: Wraps the introductory elements of a website at the top of a page or section. This could be things like logos, main headings, or search bars.
- **<nav>**: Used for blocks of navigation links to help users get around a site.
- **<main>**: Holds the main content of your webpage, search engines specifically look for it when they crawl a site.
- **<footer>**: usually goes at the bottom of a page or section and usually holds copyright details or contact info

Exercise 5

```
const numbers = [1-3];  
const newNumbers = [...numbers, 4];  
const user = { name: "Meritxell", age: 30, city: "BCN" };  
const { name, age } = user;  
const newUser = { name: "Arnau", age: 24, city: "GI" };  
const { name: userName, ...remaining } = newUser;
```

- ① Declare a constant called **numbers** and assign it an array literal with three integers. By using **const** we make sure the variable cannot be reassigned to something else.
- ② We use the spread operator **...** to create a new array, using the elements of **numbers** and unpacking them into **newNumbers** and adding a 4 at the end.
- ③ Declare a constant **user** and assign it a dictionary of key-value pairs.
- ④ We use an object destructuring assignment (Brown 2016, chap. 5). Instead of having to write **user.name** and **user.age**, this syntax pulls those properties directly out of the **user** object and assigns them to local variables with the same names.
- ⑤ New object **newUser**.
- ⑥ Destructure **newUser**, rename the **name** property to a new variable called **userName**. After that, the **...** syntax gathers the remaining properties which are **age** and **city**, and packages them into a new object called **remaining**.

Exercise 6

Both of these tools help control how often a function executes during rapid events, but they handle the execution in different ways

Debounce delays running a function until some time has passed since the last trigger, i.e. the timer resets if the event fires again. Throttle instead forces a function to run at most once during a set time window, ignoring any extra triggers that happen in between (Saafan 2024).

It is a good idea to use debounce when you are only in the final result and need to wait for a quiet period.

In (Saafan 2024), the author used this for a class project with a search input field. Instead of calling your backend API on every single keystroke, he chose to debounce the input to fetch data only after the user pauses typing. Throttle on the other hand is what you need when you want real-time updates but have to limit their frequency to avoid crashing the browser. Infinite scrolling is a perfect use case for this: as users scroll down the page, throttle ensures the code checks the scroll position at a steady rate instead of hundreds of times per second.

Exercise 7

`null` and `undefined` both denote the absence of a value but play different roles in JavaScript. `undefined` indicates that a variable has been declared but has not yet been assigned a value, and `null` shows that its value is completely unknown or does not exist (Brown 2016, chap. 3).

`NaN` stands for “not a number” and is more a placeholder rather than a mathematical value. It appears when computations or conversions fail to produce a valid double-precision floating-point number (Brown 2016, chap. 3).

The strict equality operator (`===`) verifies that the data type and the value are perfectly identical. IN this case `null` and `undefined` are distinct data types, so the comparison yields as false. Abstract equality (`==`) allows for type coercion, meaning JS attempts to convert differing types to match before evaluating them (Brown 2016, chap. 5).

`NaN` behaves in a different way⁰. It is completely unique because it is never equal to anything, not even itself. Expressions like `NaN === NaN` and `NaN == NaN` will always return false. To accurately determine if a numeric value is `NaN`, programmers must rely on the built-in `isNaN()` function (Brown 2016, chap. 5).

Exercise 8

`a` and `b`, are two separate array objects in memory. `a` is built using standard array literal syntax whereas `b` relies on the `Array` constructor. Since arrays in JS are just a special type of object, they follow standard object comparison rules (Brown 2016, chap. 3).

Checking `a === b` evaluates to `false` because the operator checks if two variables point to the exact same object in memory (Brown 2016, chap. 5), whci is not the case. Even though `a` and `b` hold the exact same values they are different objects.

`a == b` only returns `true` if both variables refer to the exact same object (Brown 2016, chap. 5). As we said before, the abstract equality operator does type coercion, but still yields false in this case as `a` and `b` are different objects

Exercise 9

CSS Grid is a two-dimensional layout system that handles rows and columns at the same time while Flexbox is a one-dimensional system handling either a row or a column ([Rai 2026](#)). Grid works from the layout in by establishing a strict structure first, whereas Flexbox works from the content out by distributing space based on the size of the content ([MDN Web Docs 2025d](#)).

CSS Grid should be used for macro layouts like overall page structures, and Flexbox for smaller layouts like navigation bars or aligning content inside individual components ([Rai 2026](#)).

Example HTML code>

```
<html>
<body>
  <h2>Flexbox vs CSS Grid Layout</h2>
  <div class="container">
    <div class="card">Card 1</div>
    <div class="card">Card 2</div>
    <div class="card">Card 3</div>
  </div>
</body>
</html>
```

Flexbox css code:

```
.container {
  display: flex;
  gap: 10px;
}

.card {
  flex: 1;
}
```

CSS Grid css code:

```
.container {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;
  gap: 10px;
}
```

For a layout of evenly spaced cards, CSS Grid is more appropriate. Flexbox requires styling both the parent container and adding `flex: 1` to every child element to maintain even sizing. With CSS Grid you only need to define the columns on the parent container and the child elements will follow the parent structure.

Exercise 10

```
const arr = [10, 20, 30];
arr.foo = "bar";
console.log(arr.length);
console.log(arr.foo);
```

The output of running the script is the following:

```
3
"bar"
```

In this snippet we first define an array `arr` of three elements. We then create a string-based property `foo`. When using the `length` property of the array we get the indexed numeric elements, ignoring any other properties outside of the array. That is why the length is still 3 after adding the `foo` property.

In JS arrays are natively defined as a special type of object, and because they are built on top of objects, arrays allow you to assign non-numeric or arbitrary keys to them, though doing so contradicts their intended purpose and is generally discouraged ([Brown 2016, chap. 3](#)).

Exercise 11

- `Object.keys()` grabs the property names from an object and hands them back as an array of strings ([MDN Web Docs 2022a](#)).
- `Object.values()` does the exact opposite: pulls out the values assigned to those properties and returns them in an array ([MDN Web Docs 2022b](#)).
- `Object.entries()` does both both of these approaches: returns an array of arrays, where each inner array represents a specific key-value pair from your object ([MDN Web Docs 2025c](#)).

```
const obj = {
  foo: "bar",
  num: 42
};

console.log(Object.keys(obj));
console.log(Object.values(obj));
console.log(Object.entries(obj));
```

Output:

```
["foo", "num"]
["bar", 42]
[["foo","bar"],["num", 42]]
```


Exercise 12

The output of the code will be a new array with the results of the `map` operation on each of the elements of the original one, without modifying the original array itself ([Brown 2016, chap. 8](#)).

The `map()` method transforms array elements and returns a new array containing the results of the callback function, while `forEach()` returns `undefined` and is used to execute a provided callback function once for each array element `forEach()` is not chainable whereas the new array returned by `map()` can be chained with other array methods ([MDN Web Docs 2025b](#)).

For example, iterating through a collection of items to update their properties directly is a perfect use case for `forEach()`. However calling `map()` when you do not intend to use the returned array is considered an anti-pattern since it wastes resources building a new data structure ([MDN Web Docs 2025b](#)):

```
const products = [
  { name: "sports car" },
  { name: "laptop" },
  { name: "phone" }
];

products.forEach((product) => {
  product.price = 100;
});
```

Exercise 13

The value of `{ msg }` is not explicitly defined in the file, it will display whatever string is passed down by its parent component. `msg` appears on the file as a prop but does not define its value as it is meant to dynamically be received by a parent component.

The type is `String`, as messages usually are displayed as such. This type is indicated so that Vue can use that information for type validation.

Scoped in the `<style>` sections indicates that that specific rule should only apply to the elements within this component.

Exercise 14

`main.js` serves as the primary JavaScript entry point for the application. It creates the application instance using the `createApp` function, imports the root component and mounts the entire application to an HTML DOM element ([Vue.js 2026b](#)).

`App.vue` is the root Single-File Component of the application. It acts as the top-level container for the project. Every Vue app requires a root component that encapsulates the primary layout and logic and it can

contain other reusable components as its children to form a nested component tree ([Vue.js 2026a](#)) ([Vue.js 2026b](#)).

`vite.config.js` is the configuration file for the development server. It allows to adapt Vite to specific project needs. You can configure framework support via plugins, modify browser target settings, specify other root directories, and define behavior for production build ([Vite.dev 2026](#)).

Exercise 15

It is possible to use CSS preprocessors such as SCSS or Less inside the styles of a `.vue` file. Once the dependencies are installed, just add a `lang` attribute to the style tag ([Vue.js 2026d](#)). Examples:

- `<style lang="scss">`
- `<style lang="less">`

Adding the `setup` keyword to a `<script>` declaration a shortcut is a compile-time syntactic sugar used in Vue 3 to author components using the Composition API ([Vue.js 2026c](#)).

In the older Composition API structure, you had to export a default object, define a `setup()` method and manually return every variable, import or function you want to show up in your template ([LearnVue 2026](#)). In Vue 3, it automatically exposes all your top-level bindings to the template ([LearnVue 2026](#)).

Exercise 16

I can add my name to the file manually by using any text editor and adding the `<h2>` header within the template right after the `<h1>`, which is good practice.

Exercise 17

To add a new component to the `App.vue` file we must follow these steps:

1. Inside the `<script setup>` block, use an `import` statement to bring the child component into the parent file. Any components imported into this scope are automatically exposed and made available directly to the template; there is no need to manually register the component inside a separate `components` object ([Vue.js 2026a](#)).
2. Once imported you can place the component directly inside the `<template>` block as a custom HTML tag.

This is the result of adding a `NewComponent.vue` file to the original `App.vue` file:

```
<script setup>
import HelloWorld from './components/HelloWorld.vue'
import NewComponent from './components/NewComponent.vue'
</script>
```

```
<template>
  <HelloWorld />
  <NewComponent />
</template>
```

Exercise 18

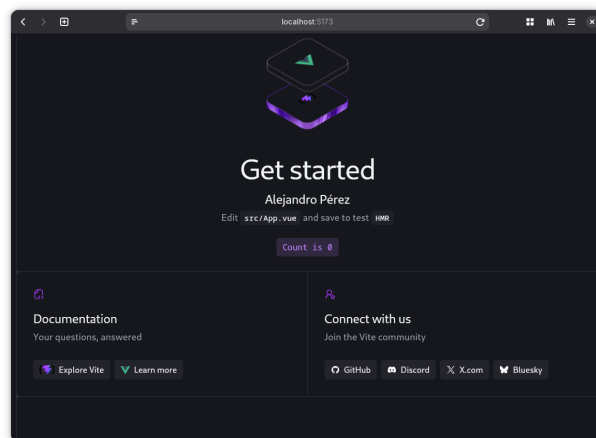
To get a development environment running `npm run dev` in the root of the project, and to prepare a production application running `npm run build` ([Vite.dev 2026](#)).

Exercise 19

```
> cd test
index.html node_modules package.json package-lock.json public README.md src vite.config.js
> npm run dev
> test@0.0.0 dev
> vite

VITE v8.0.0 ready in 163 ms
➤ Local:   http://localhost:5173/
➤ Network: use --host to expose
➤ press h + enter to show help
```

(a) `npm run dev` command



(b) Vue App running locally

Figure 1: Running Vue Application

References

- Adobe. 2023. “Single-Page Applications (SPAs) What They Are and How They Work.” <https://business.adobe.com/blog/basics/learn-the-benefits-of-single-page-apps-spa>.
- Batsou, Eleftheria. 2025. “Preview of Vue 3.6 & Vapor Mode: Insights from Evan You’s Vue.js Nation 2025 Talk.” Vue School. March 17, 2025. <https://vueschool.io/articles/news/vn-talk-evan-you-preview-of-vue-3-6-vapor-mode/>.
- Brown, Ethan. 2016. *Learning JavaScript*. 3rd ed. O’Reilly Media.
- freeCodeCamp. 2019. “Semantic HTML5 Elements Explained.” freeCodeCamp. 2019. <https://www.freecodecamp.org/news/semantic-html5-elements-explained/>.
- GeeksforGeeks. 2026. “ReactJS Virtual DOM.” GeeksforGeeks. January 10, 2026. <https://www.geeksforgeeks.org/reactjs-virtual-dom/>.
- LearnVue. 2026. “Explaining the New Script Setup Type in Vue 3 - RFC Takeaways.” LearnVue. 2026. <https://learnvue.co/>.
- MDN Contributors. 2025. “SPA (Single-Page Application): Glossary.” Mozilla. 2025. <https://developer.mozilla.org/en-US/docs/Glossary/SPA>.
- MDN Web Docs. 2022a. “Object.keys() - JavaScript.” Mozilla. 2022. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/keys.
- . 2022b. “Object.values() - JavaScript.” Mozilla. 2022. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/values.
- . 2025a. “ARIA: Landmark Role.” Mozilla. 2025. https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Roles/landmark_role.
- . 2025b. “Array.prototype.forEach() - JavaScript.” Mozilla. 2025. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/forEach.
- . 2025c. “Object.entries() - JavaScript.” Mozilla. 2025. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object/entries.
- . 2025d. “Relationship of Grid Layout to Other Layout Methods.” Mozilla. 2025. https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Grid_Layout/Relationship_of_Grid_Layout.
- Montti, Roger. 2024. “What Semantic HTML Is and Why It’s Good for SEO.” Search Engine Journal. 2024. <https://www.searchenginejournal.com/semantic-html-seo/>.
- Next.js. 2026. “Server-Side Rendering (SSR).” Vercel. March 16, 2026. <https://nextjs.org/docs/pages/building-your-application/rendering/server-side-rendering>.
- Omoyeni, Timi. 2024. “The Future of Vue: Vapor Mode.” Vue Mastery. April 25, 2024. <https://www.vuemastery.com/blog/the-future-of-vue-vapor-mode/>.
- OutSystems. 2025. “Single Page App: SPA Frameworks & Their Role in Development.” OutSystems. 2025. <https://www.outsystems.com/application-development/spa-single-page-app-defined-with-examples/>.
- Pavlik, Vlado. 2025. “What Is Semantic HTML? And How to Use It Correctly.” Semrush. 2025. <https://www.semrush.com/blog/semantic-html/>.
- Pluralsight. 2019. “Pros and Cons of Client-Side Rendering.” December 19, 2019. <https://www.pluralsight.com/resources/blog/guides/pros-and-cons-of-client-side-rendering>.
- Rai, Kriti. 2026. “CSS Grid Vs Flexbox: When to Use What.” 2026. <https://www.kritirai.com/blog/when->

[to-use-what-css-grid-vs-flexbox](#).

Rüsch, Lukas. 2025. “Vue Vapor: Goodbye Virtual DOM, Hello Performance!” BLUESHOE. May 9, 2025. <https://www.blueshoe.io/blog/vue-vapor-performance-without-virtual-dom/>.

Saafan, Amr. 2024. “JavaScript Performance Optimization: Debounce Vs Throttle Explained.” DEV Community. 2024. <https://dev.to/nilebits/javascript-performance-optimization-debounce-vs-throttle-explained-5768>.

Shopify. 2025. “SSR Vs. CSR: Server-Side Vs. Client-Side Rendering Explained.” 2025. <https://www.shopify.com/blog/ssr-vs-csr>.

SolutionsHub, EPAM. 2024. “What Is Server-Side Rendering: Pros and Cons.” 2024. <https://solutionshub.epam.com/blog/post/what-is-server-side-rendering>.

Vite.dev. 2026. “Getting Started.” Vite.dev. 2026. <https://vite.dev/guide/>.

Vue.js. 2025. “Rendering Mechanism.” Vue.js. 2025. <https://vuejs.org/guide/extras/rendering-mechanism.html>.

———. 2026a. “Component Basics.” Vue.js. 2026. <https://vuejs.org/guide/essentials/component-basics.html>.

———. 2026b. “Creating a Vue Application.” Vue.js. 2026. <https://vuejs.org/guide/essentials/application>.

———. 2026c. “<Script Setup>.” Vue.js. 2026. <https://vuejs.org/api/sfc-script-setup.html>.

———. 2026d. “Using Pre-Processors.” Vue.js. 2026. <https://vue-loader.vuejs.org/guide/pre-processors.html>.